

SE3001-

Software construction and Development

ENGR.MUHAMMAD UMER HAROON

- **What is XML?**
- **XML** stands for e**X**tensible **M**arkup **L**anguage. It is a language (not> a programming language) that uses the markup and can extend. It is derived from **S**tandard **G**eneralized **M**arkup **L**anguage(SGML). XML also uses DTDs (**D**ocument **T**ype **D**efinitions) to define the XML document structure.

Engr.Muhammad Umer Haroon

- XML is not for handling computational operations and algorithms. Thus, XML is not a programming language.
- The main goal is to transport data not to display information. XML bridges the gap between human readability and machine readability. Unlike HTML tags, XML tags are self-descriptive.

Engr.Muhammad Umer Haroon

XML Features

- Here are some important features of XML:
- It is extensible and human-readable.
- It is platform and language independent.
- It preserves white space.
- Overall simplicity.
- XML tags are not predefined. You need to define your customized tags.
- XML was designed to carry data, not to display that data.
- Well-structured format is easy to read and write from programs.

Engr.Muhammad Umer Haroon

XML Encoding

Encoding is the conversion of Unicode characters to their binary representation. UTF is use for XML encoding. **UTF** stands for **U**CS (**UCS** stands for **U**niversal **C**haracter **S**et) **T**ransformation **F**ormat. Mainly, there are two types of UTF encoding.

1.UTF-8 : UTF-8 uses 8-bits to represent the characters.

Example:

```
<?xml version="1.0" encoding="UTF-8"?>
```

2.UTF-16

It uses 16-bits to represent the characters.

Example:

```
<?xml version="1.0" encoding="UTF-16"?>
```

You can use encoding inside the XML declaration. UTF-8 is the default encoding in XML.

Engr.Muhammad Umer Haroon

XML Syntax

The below code segment shows the basic XML syntax.

```
<?xml version = "1.0" encoding = "UTF-8" ?>
 <root>
       <child>
               <subchild>
              .....</subchild>
       </child>
 </root>
```

XML Declaration

XML declaration consists of the XML version, character encoding or/and standalone status. The declaration is optional.

Syntax for XML Declaration

The below code segment shows the syntax for XML declaration.

```
<?xml version="version_number," encoding="character_encoding" standalone="yes_or_no" ?>
```

Engr.Muhammad Umer Haroon

XML Declaration Rules

Following are XML declaration rules.

- If the XML declaration is present, it must be the first thing that appears.
- The XML declaration is case sensitive, and it must start with the lowercased **<?xml**.
- It has no closing tag.

Example of XML Declaration

Following code segment shows an example of an XML declaration.

```
<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
```

Engr.Muhammad Umer Haroon

XML Comments

Comments are optional. Adding comments help to understand the document content.

Syntax for XML Comments

A comment begins with **<!–** and ends with **–>**.
Following code segment shows the syntax for XML comments.

```
<!-- Add your comment here -->
```

Engr.Muhammad Umer Haroon

XML Tags and Elements

Tags work as pairs except for declarations. Every tag pair consists of an **opening tag** (also known as the **start tag**)
and a **closing tag** (also known as the **end tag**).
Tag names are enclosed in **<>**. For a particular tag pair, the start and end tags must be identical except the end tag has **/** after the **<**.

```
<name>...</name>
```

Engr.Muhammad Umer Haroon

Anything between the opening and closing tags is referred to as **content**.

Opening tag, content, and closing tag, altogether, is referred to as an **element**.

```
Opening tag + content + closing tag = an element
```

Note: Elements may also contain attributes.

```
<age>20</age>
```

Engr.Muhammad Umer Haroon

```
<age>20</age>
```

In the above element,

- **age** is the **name of the element.**

Note: Tag name also referred to as an **element** or **element name**.

- **<age>** – opening tag
- **25** – content
- **</age>** – closing tag.

If there is no content between the tags, as shown below, it referred to as **empty tags**.

```
<result></result>
```

Engr.Muhammad Umer Haroon

XML Tag and Element Rules

Following list shows XML tag and element rules.

- Tags are case sensitive.

Correct:

```
<age>20</age>
```

Wrong:

```
<age>20</Age>
```

Engr.Muhammad Umer Haroon

- **Note: AGE**, **Age,** and **age** are three different names in XML.
- All XML documents must contain a single root element.
- All elements must have a closing tag (except for declarations).
- A tag name must begin with a letter or an underscore, and it cannot start with the **XML.**
- A tag name can contain letters, digits, hyphens, underscores, and periods. Hyphens underscore, and periods are the only punctuation marks allowed.
- A tag name cannot contain spaces.
- All elements must be nested properly.

Engr.Muhammad Umer Haroon

<book>
<book-title>
<_data>
<data_1>
<item.name>
<1book>
<-title>
<.data>

Engr.Muhammad Umer Haroon

Correct nesting

```
<parent>
    <child>Content</child>
</parent>
```

Engr.Muhammad Umer Haroon

Incorrect nesting

```
<parent>
    <child>Content</parent>
</child>
```

Engr.Muhammad Umer Haroon

Example:

```
<b><u>This text is bold and italic</u></b>
<b><u>This text is bold and italic.</b></u>
```

Engr.Muhammad Umer Haroon

XML Attributes

Attribute for an element is placed after the tag name in the start tag. You can add more than one attribute for a single element with different attribute names.

Engr.Muhammad Umer Haroon

Let’s consider the below XML document.

```
<company name="ABC Holdings" location="London">
<chairman>Mr. John</chairman>
 <gm>Mr. Wood</gm>
</company>
```

Engr.Muhammad Umer Haroon

- There are two attributes in the **company** element, i.e. **name** and **location**.
- Let’s study the **name** attribute,
- **name=”ABC Holdings”** – an attribute
- **name** – attribute name / attribute
- **ABC Holdings** – attribute value
- the **company** is the **root** element.

Engr.Muhammad Umer Haroon

XML Attribute Rules

- Attribute values must be within quotes.
- An element cannot contain several attributes with the same name.

Engr.Muhammad Umer Haroon

Attribute versus Element

Here is another example.

Let’s consider documents A and B given below.

Document A:

```
<teacher subject="English"> <name>Mr. John</name>.
<qualification>Graduate</qualification> </teacher>
```

Document B:

```
<teacher> <subject>English</subject> <name>Mr. John</name>
<qualification>Graduate</qualification> </teacher>
```

In document A, the **subject** is an **attribute.**

In document B, the **subject** is an **element**.

Engr.Muhammad Umer Haroon

XML Entities

Engr.Muhammad Umer Haroon

- **What are XML Entities?**
- In simple terms, entities are a way of representing special characters. Entities are also known as **entity references**.
- **Why You Need XML Entities?**
- Some characters (such as “, **& <**, and so on) are reserved in XML. They are referred to as **special characters** and cannot be directly used for other purposes.
- For example, the **<** and **>** symbols a used for **tags**. You cannot directly type from the keyboard for **less than** and **greater than** signs. Instead, you need to use entities.

Engr.Muhammad Umer Haroon

Following table shows some of the popular XML entities.

Example:

Character	Description	Entity Name	Usage
“	Quotation mark (double quote)	**quot**	"
&	Ampersand	**amp**	&
‘	Apostrophe (single quote)	**apos**	'
<	Less than sign	**lt**	<
>	Greater than sign	**gt**	>

Engr.Muhammad Umer Haroon

Example:

```
<friend>
<name>My friends are Alice &amp;amp; Jane.
</name>
</friend>
```

Engr.Muhammad Umer Haroon

JSON versus XML

Similarities between JSON and XML

The below list shows the similarities between JSON and XML.

- Both are open formats.
- Both are self-describing.
- Both have a hierarchical structure.
- Both can parse and use by several programming languages.

Engr.Muhammad Umer Haroon

Differences between JSON and XML

Engr.Muhammad Umer Haroon

Read your self

JSON	XML	
Stands for	JavaScript Object Notation	Extensible Markup Language
Extended from	JavaScript	SGML
Data storage	Data stored as key-value pairs.	Data stored as a tree structure.
Namespaces	No support for namespaces.	Supports namespaces.
Comments	Adding comments is not supported.	Can add comments.
Data accessibility	Readily accessible as JSON objects.	Data need to be parsed.
Metadata	Adding metadata is not supported.	Can write metadata.
Types	JSON types: string, number, array, Boolean.	All XML data should be strings.
Data types of support	Supports **text** and **numbe r** data types only.	Support many data types (text, numbers, images, so on)

Engr.Muhammad Umer Haroon

Read your self

Array's support	More support for arrays compared to XML.	No or less support for arrays.
Object's support	Native support for object.	The object has to be express by conventions.
AJAX toolkit support	Supported.	Not fully supported.
Retrieving values	Easy.	Difficult.
Deserializing/serializing	Fully automated.	Developers have to write JavaScript code.
Browser support	Supported by most browsers.	Cross-browser XML parsing can be tricky.
Encoding	Only supports UTF-8 encoding.	It supports various encoding.
Display capabilities	No display capabilities.	Offer display capabilities.
Document size	Smaller than XML.	Large than JSON.
Filename Extension	**.json**	**.xml**
Security	Less secured.	More secure than JSON.
Easy to read	Relatively easy.	Relatively difficult.
Learning curve	Easy to learn.	Steep learning curve.
Website	https://www.json.org/json-en.html	https://www.w3.org/TR/xml11/

Engr.Muhammad Umer Haroon

Basic JSON Syntax

```
{string:value, .......}
```

Same example with JSON and XML

With JSON

```
{"books":[ {"name":"Anna Karenina", "author":"Leo Tolstoy"},
{"name":"One Hundred Years of Solitude", "author":"Gabriel Garcia
Marquez"}, {"name":"The Great Gatsby", "author":"Scott Fitzgerald"},
{"name":"Invisible Man", "author":"Ralph Ellison"} ]}
```

Engr.Muhammad Umer Haroon

With XML

```
<?xml version = "1.0" encoding = "UTF-8" ?>
 <books> <book>
<name>Anna Karenina</name> <author>Leo Tolstoy</author> </book>
 <book> <name>One Hundred Years of Solitude</name> <author>Gabriel
Garcia Marquez</author> </book>
 <book> <name>The Great Gatsby</name> <author>Scott Fitzgerald</author>
</book>
 <book> <name>Invisible Man</name> <author>Ralph Ellison</author>
</book> </books>
```

Engr.Muhammad Umer Haroon

XML DTD

- **What is DTD?**
- DTD stands for **D**ocument **T**ype **D**efinition. It defines the structure of an XML document using some legal elements. XML DTD is optional.
- **DTD Rules**
- Following list shows DTD rules.
- If DTD is present, it must appear at the start of the document (only the XML declaration can appear above the DTD).
- The element declaration must start with an **!** mark.
- The DTD name and element type of the root element must be the same.

Engr.Muhammad Umer Haroon

Example of an internal DTD:

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE student [ <!ELEMENT student(firstname,lastname,school)>
<!ELEMENT firstname (#PCDATA)>
<!ELEMENT lastname (#PCDATA)>
<!ELEMENT school (#PCDATA)> ]>
<student>
      <firstname>Mark</firstname>
      <lastname>Wood</lastname>
      <school>Hills College</school>
 </student>
```

Engr.Muhammad Umer Haroon

- In the above example,
- **!DOCTYPE student** indicates the beginning of the DTD declaration. And the **student** is the root element of the XML document.
- **!ELEMENT student** indicates the **student** element must contain **firstname, lastname** and **school** elements.
- **!ELEMENT firstname** indicates the **firstname** element is of type **#PCDATA** (**P**arsed **C**haracter **Data**).
- **!ELEMENT lastname** indicates the **lastname** element is of type **#PCDATA**.
- **!ELEMENT school** indicates the **school** element is of type **#PCDATA**.

Engr.Muhammad Umer Haroon

The DTD file content (student.dtd) as follows.

```
<!ELEMENT student (firstname,lastname,school)>
<!ELEMENT firstname (#PCDATA)>
<!ELEMENT lastname (#PCDATA)>
<!ELEMENT school (#PCDATA)>
```

Engr.Muhammad Umer Haroon

- **XML DOM**
- **What is DOM?**
- DOM stands for **D**ocument **O**bject **M**odel. It defines a standard manner of accessing and manipulating XML documents. DOM has a (hierarchical) tree structure.

Engr.Muhammad Umer Haroon

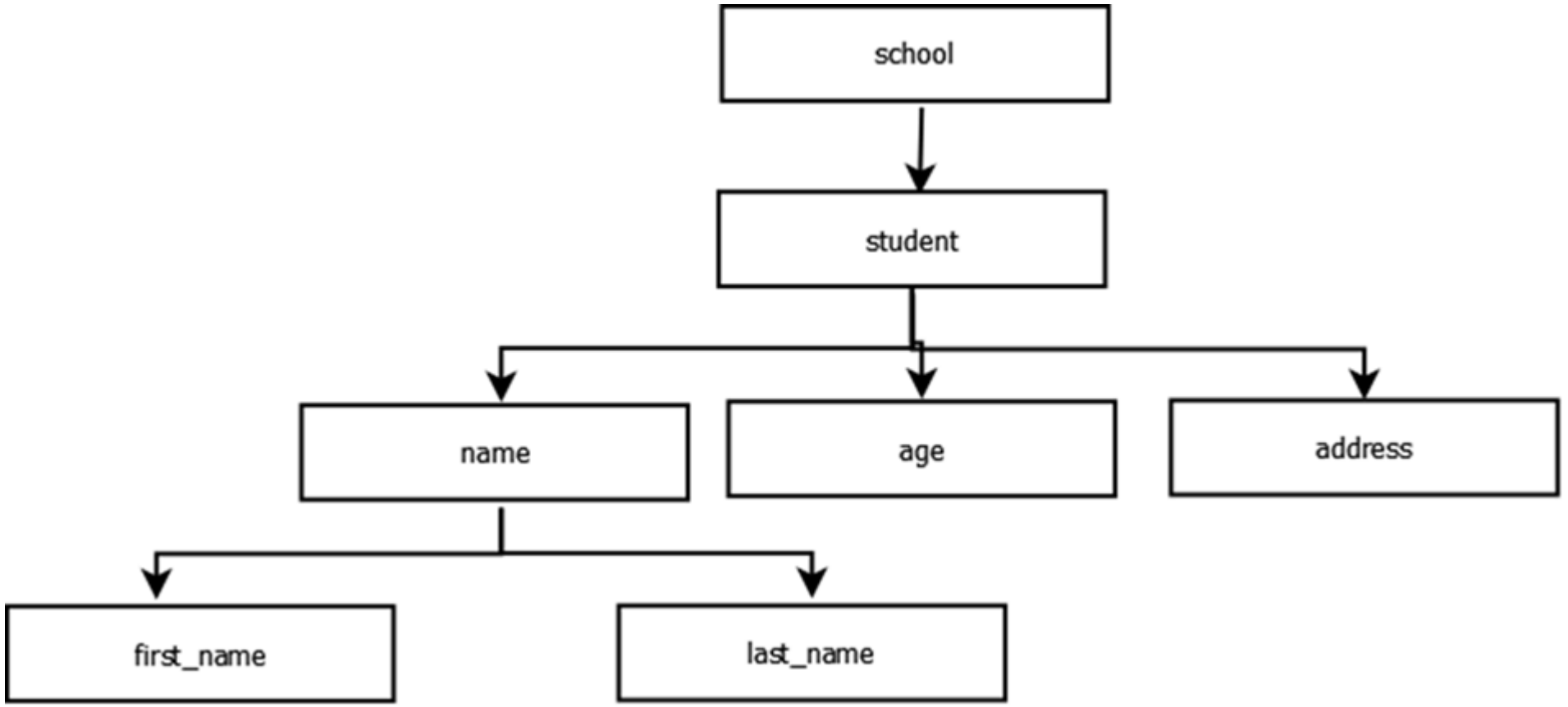
example of DOM

Let’s consider the below XML document.

```
<?xml version="1.0" encoding="UTF-8" ?>
 <school>
 <student>
<name>
<first_name>Alex</first_name>
 <last_name>Clarke</last_name>
</name>
 <age>14</age>
<address>No. 35, Flower Road, Leeds</address>
 </student>
</school>
```

Engr.Muhammad Umer Haroon

XML DOM Example



Engr.Muhammad Umer Haroon

XML Validation

What are Well-formed XML Documents?
Well-formed XML documents are XML documents with correct syntax.
What are Valid XML Documents?
Valid XML documents are well-formed and also conform to the DTD rules.

Engr.Muhammad Umer Haroon

XML Editors

- Search them some are open source , other are paid , some might be graphical

Engr.Muhammad Umer Haroon

XML Schemas

XML **schema** (also known as **XML schema definition** or **XSD**) use to describe the XML document structure. It is an alternative to DTD.

Why Schema is Important?

DTD is not powerful as schema as it is not extensible and flexible enough. So, it may not be suitable for some situations. In such a situation schema is important. The main purpose of using XML schema is to define the elements and attributes of an XML document.

Engr.Muhammad Umer Haroon

Advantages of XML

Here, pros/benefits of XML:

•It made it easy to transport and share data.

•XML improves the exchange of data between various platforms.

•It is a markup language, which is a set of characters or/and symbols placed in a text document.

•XML indicates how the XML document should look after it is displayed.

•It simplifies the platform change process.

•It enhances data availability.

•It supports multilingual documents and Unicode.

•Provide relatively easy to learn and code.

•It is a markup language, which is a set of characters or/and symbols placed in a text document.

•It performs validation using DTD and Schema.

•Makes documents transportable across systems and applications. With the help of XML, you can exchange data quickly between different platforms.

•XML separates the data from HTML.

Engr.Muhammad Umer Haroon